

date: 2026-02-18

SPEC-001: zetl — Bi-directional Link Graph CLI

Information Table

Field	Value
Document ID	SPEC-001
Title	zetl — Bi-directional Link Graph CLI
Version	0.1.0
Status	Draft
Author	Agent (USDD Protocol v1.0.0)
Date	2026-02-18
Audience	Agent, Human
Trace	USDD Agent Protocol v1.0.0

1. Overview

zetl is a lightweight, agent-friendly CLI tool for personal knowledge management. It parses `[[wikilink]]` bi-directional links from a corpus of Markdown files, builds an in-memory graph of page relationships, and exposes query and validation commands over that graph. It is designed to be invoked by AI agents as part of a specification-driven development workflow, and by humans managing a Zettelkasten-style knowledge base.

1.1 Design Philosophy

- Files are the source of truth.** zetl never modifies user Markdown files. It reads, indexes, and reports.
- Cross-application compatible.** Wikilink syntax follows the Obsidian convention, which is the most widely adopted and is readable by Logseq, Foam, Dendron, and others.
- Agent-first, human-friendly.** All output is structured (JSON by default) for machine consumption, with a human-readable table mode for interactive use.
- Fast and disposable.** The index is a cache, not a database. It can be rebuilt from scratch in seconds for typical vaults (< 10,000 files).
- Zero configuration to start.** Point it at a directory and go.

1.2 Scope

In scope:

- Wikilink parsing (`[[...]]` syntax family)
- Directed graph construction (forward links and computed backlinks)

- Graph queries (backlinks, forward links, orphans, dead links, connected components, shortest path)
- Syntax validation and broken-link detection
- Fuzzy page-name matching via SimHash + Hamming distance
- Structured output (JSON, table)

Out of scope:

- File editing, creation, or modification
- Full-text search (defer to `rg`, `grep`, or dedicated tools)
- GUI or TUI
- Markdown rendering
- Sync, collaboration, or multi-user features
- Tag extraction (future SPEC)

2. User Profiles

2.1 Agent Operator

Role: AI coding/research agent (e.g., Claude Code, Aider, custom MCP tool)

Goals:

- Create documents with `[[wikilinks]]` **and** validate them
- Query the link graph **to** discover related pages
- Check **for** broken links before committing
- **Find** semantically similar **page** names **to** avoid duplication

Constraints:

- Requires structured (JSON) output **for** parsing
- Invokes CLI commands non-interactively
- May **run in** CI/CD pipelines

Daily workflow:

1. Create/**edit** markdown files with `[[wikilinks]]`
2. **Run** ``zetl check`` **to** validate link integrity
3. **Run** ``zetl backlinks <page>`` **to** discover context
4. **Run** ``zetl similar <query>`` **to find** near-duplicates before creating new pages

2.2 Human Knowledge Worker

Role: Researcher, writer, or developer maintaining a personal knowledge base

Goals:

- Understand the structure of their knowledge graph
- Find orphaned or disconnected notes
- Audit broken links
- Explore relationships between topics

Constraints:

- Prefers human-readable table output
- May not be deeply technical
- Works from the terminal alongside Obsidian, Logseq, or a text editor

Daily workflow:

1. Write notes in Obsidian/Logseq/editor
 2. **Run** ``zetl stats`` for an overview
 3. **Run** ``zetl orphans`` to find disconnected notes
 4. **Run** ``zetl dead-links`` to clean up broken references
-

3. Wikilink Syntax Specification

zeti follows the **Obsidian wikilink convention**, which is the de-facto cross-application standard. This ensures compatibility with Obsidian, Logseq (in Markdown mode), Foam, Dendron, and other tools that read `[[...]]` syntax.

3.1 Supported Link Formats

Pattern	Semantics	Example
<code>[[Page Name]]</code>	Link to page	<code>[[Zettelkasten Method]]</code>
<code>[[Page Name Display Text]]</code>	Link with alias	<code>[[Zettelkasten Method ZK]]</code>
<code>[[Page Name#Heading]]</code>	Link to heading within page	<code>[[Zettelkasten Method#History]]</code>
<code>[[Page Name#Heading Alias]]</code>	Heading link with alias	<code>[[Zettelkasten Method#History bg]]</code>
<code>[[Page Name^block-id]]</code>	Link to block	<code>[[Zettelkasten Method^abc123]]</code>
<code>![[Page Name]]</code>	Embed / transclude (parsed as link)	<code>![[Diagram of ZK]]</code>
<code>[[#Heading]]</code>	Link to heading in current page	<code>[[#See Also]]</code>

3.2 Page Name Resolution

Resolution follows Obsidian's shortest-unambiguous-path convention:

1. **Exact match:** case-insensitive filename match (ignoring `.md` extension)
2. **Normalized match:** spaces, hyphens, and underscores treated as equivalent (`my-page` = `my_page` = `my page`)
3. **Path-qualified match:** if the link contains `/`, match against relative path from vault root
4. **Ambiguous match:** if multiple files match, report as a warning (do not silently pick one)

3.3 Parsing Rules

- Links are extracted from Markdown body text only
- Links inside fenced code blocks (`````) SHALL be ignored
- Links inside inline code (```) SHALL be ignored
- Links inside HTML comments (`<!-- -->`) SHALL be ignored
- Links inside YAML frontmatter (`---` delimiters) SHALL be ignored
- The `#heading` and `^block-id` suffixes are stored as metadata but the **page name** (before `#` or `^`) is the graph edge target
- Embed syntax (`![[...]]`) is treated as a link for graph purposes, with an `embed: true` flag

3.4 Formal Grammar

```
wikilink      ::= embed? '[' target ( '|' alias )? ']'
embed         ::= '!'
target        ::= page-ref ( heading-ref | block-ref )?
page-ref      ::= [ '^' [ '\ ' ] '#' ]+
heading-ref   ::= '#' [ '^' [ '\ ' ] ]+
block-ref     ::= '^' [ 'a-zA-Z0-9-' ]+
alias         ::= [ '^' [ '\ ' ] ]+
```

4. Requirements

4.1 Functional Requirements

REQ-001: Index Markdown Files

The **system** SHALL recursively scan a specified directory **for** files matching the glob pattern `**/*.md` **and parse all** wikilinks **from each** file **WITHIN 5** seconds **for** a corpus of **10,000** files **FOR all user** roles **WITH** the resulting **index** containing **every** wikilink occurrence **with** its source file, line **number**, column, **and** parsed components (page, heading, block, alias, embed flag).

Trace:

- TEST-001
- CON-001

REQ-002: Build Link Graph

The **system** SHALL construct a directed graph **where**:

- **Each** node **is** a page (resolved filename **or** unresolved page name)
- **Each** edge **is** a link **from** source page **to** target page
- Edges carry metadata: line **number**, alias, heading, block-ref, embed flag
- Backlinks are computed **as** the reverse of forward links

FOR all user roles **WITH** the graph queryable **via** CLI subcommands.

Trace:

- TEST-002
- CON-002

REQ-003: Query Forward Links

The system SHALL **return all** pages that a given page links **to**, including link metadata (**line**, **alias**, heading, **block**), **FOR all user** roles **WITH** output **in** JSON (**default**) **or** table format.

Trace:

- TEST-003
- CON-003

REQ-004: Query Backlinks

The **system** SHALL **return** all pages that link **to** a given page, including **the** source **file**, **line number**, **and** link context (surrounding **text**), **FOR all user** roles **WITH** output **in** JSON (default) **or** table **format**.

Trace:

- TEST-004
- CON-003

REQ-005: Detect Dead Links

The **system** SHALL identify all wikilinks whose target page does **not** **resolve to any** existing **file in the** scanned **directory**,

FOR all user roles
WITH **each** dead link reported **with** source **file**, **line number**, **and** unresolved target.

Trace:
- TEST-005
- CON-004

REQ-006: Detect **Orphan** Pages

The system **SHALL** identify all pages that have **zero** incoming links (**backlinks**)
AND are not explicitly excluded,
FOR all user roles
WITH each **orphan** reported with its file path **and** forward link **count**.

Trace:
- TEST-006
- CON-004

REQ-007: Syntax Validation

The system SHALL detect and report malformed wikilink syntax including:
- Unclosed brackets: `[[page name` or `[[page name]`
- Empty links: `[[[]`
- Nested brackets: `[[page [[nested]]]]`
FOR all user roles
WITH each violation reported with file, line, column, and a diagnostic message.

Trace:
- TEST-007
- CON-004

REQ-008: Fuzzy Page Name Search via SimHash

The **system** SHALL compute a SimHash fingerprint **for each** page name (**and** optionally **the first paragraph** / frontmatter title) **and** support searching **for** pages **within a** specified Hamming distance of a query **string**,
FOR all user roles
WITH default Hamming distance threshold of **3** **and** configurable via flag.

Trace:
- TEST-008
- CON-005

REQ-009: **Graph Statistics**

The system SHALL report summary statistics including:
- Total pages (files), total links, total unique link targets
- Number of dead links, orphan pages, ambiguous links
- Most-linked-to pages (top N)
- Connected component count
FOR all user roles
WITH output in JSON (default) or table format.

Trace:
- TEST-009
- CON-006

REQ-010: Shortest Path Query

The **system** SHALL compute the shortest path (**by** link hops) **between** two named pages **in** the link graph, reporting the **full** path **of** page names, **FOR all user** roles **WITH** a "no path found" **result when pages are in** different components.

Trace:
– TEST-010
– CON-007

REQ-011: Persistent Cache

The system SHALL **cache** the parsed index **to** a file (``.zetl/index.json``) **to** avoid re-scanning unchanged files **on** subsequent invocations, **FOR all** user roles **WITH cache** invalidation based **on** file modification timestamps **AND** a ``--no-cache`` flag **to** force a **full** rescan.

Trace:
– TEST-011

REQ-012: Ignore Patterns

The system SHALL respect a ``.zetlignore`` file (gitignore syntax) at the vault root, **and** a ``--ignore`` CLI flag, **to** exclude files **and** directories from scanning, **FOR all** user roles **WITH** ``.git``, ``.node_modules``, **and** ``.zetl`` ignored by default.

Trace:
– TEST-012

4.2 Non-Functional Requirements

NFR-001: Indexing Performance

Indexing throughput SHALL be $\geq 2,000$ files/second **on** a corpus of **average**-sized Markdown files (< 10 KB each) **UNDER** single-threaded **execution on** commodity hardware **WITH** 95th percentile.

NFR-002: Query Latency

All query subcommands SHALL return results in ≤ 100 ms **UNDER** a **pre**-built index of 10,000 pages **WITH** 95th percentile.

NFR-003: Memory Usage

Peak memory consumption SHALL be ≤ 200 MB **UNDER** a corpus of 10,000 files **WITH** average file size of 5 KB.

NFR-004: Binary Size

The compiled **binary** SHALL be ≤ 10 MB (stripped, single platform) **WITH** **zero** runtime dependencies **beyond** libc.

NFR-005: Cross-Platform

The tool **SHALL** compile **and** run on Linux (x86_64, aarch64), macOS (aarch64), **and** Windows (x86_64) **WITH** identical **behaviour** across platforms (modulo path separators).

5. Architecture

5.1 Technology Choice

Language: Rust

Factor	Rationale
Performance	Zero-cost abstractions, no GC pauses, meets NFR-001/002
Binary distribution	Single static binary, no runtime deps (NFR-004/005)
Ecosystem	<code>pulldown-cmark</code> for Markdown, <code>petgraph</code> for graph, <code>clap</code> for CLI
Safety	Memory safety without GC, suitable for untrusted file input

ADR-001: Language Selection – Rust

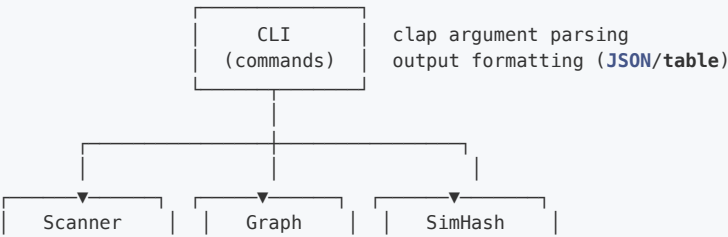
Status: Proposed

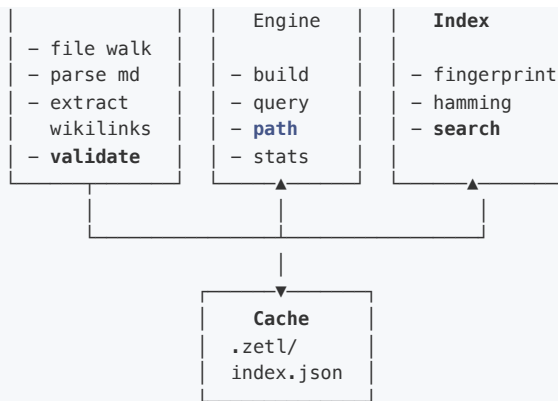
Context:
zetl is a CLI tool that must parse thousands of Markdown files quickly, **build** an **in-memory** graph, **and return** query results **with** low latency. It must ship **as** a single **binary with** no runtime dependencies.

Decision:
Implement **in** Rust **using**:
- ``clap`` **for** CLI argument parsing
- ``pulldown-cmark`` **for** Markdown event streaming (**to** identify code blocks)
- ``petgraph`` **for** the directed graph data structure
- ``serde`` / ``serde_json`` **for** structured output
- ``ignore`` crate **for** gitignore-style **path** matching
- Custom SimHash implementation **over** character n-grams **for** fuzzy search

Consequences:
+ Meets **all** NFRs (performance, **binary** size, cross-platform)
+ Strong ecosystem **for each** component
- Slower iteration speed than Python/TypeScript **for** prototyping
- Steeper learning curve **for** contributors unfamiliar **with** Rust

5.2 Component Architecture





Scanner — Walks the file tree (respecting ignore patterns), streams each Markdown file through `pulldown-cmark` to identify code/comment regions, then applies a regex over non-excluded regions to extract wikilinks. Produces a `Vec<ParsedFile>`.

Graph Engine — Consumes `ParsedFile` records, resolves page names, and builds a `petgraph::DiGraph`. Exposes query methods: forward links, backlinks, orphans, dead links, shortest path, connected components, stats.

SimHash Index — Computes 64-bit SimHash fingerprints for page names using character trigram features. Supports nearest-neighbour search by Hamming distance with configurable threshold.

Cache — Serializes the scanner output to `.zetl/index.json` with file-level mtimes. On subsequent runs, only re-parses files whose mtime has changed.

CLI — Thin layer mapping subcommands to engine calls and formatting output.

5.3 Data Model

```

/// A single extracted wikilink occurrence
struct WikiLink {
    target_page: String,      // resolved page name (normalized)
    raw_target: String,      // original text inside [[ ]]
    heading: Option<String>,  // #heading reference
    block_ref: Option<String>, // ^block-id reference
    alias: Option<String>,    // display text after |
    is_embed: bool,          // preceded by !
    line: u32,                // 1-indexed line number
    column: u32,              // 1-indexed column
}

/// Parsed result for a single file
struct ParsedFile {
    path: PathBuf,            // relative to vault root
    page_name: String,        // derived from filename (sans .md)
    links: Vec<WikiLink>,
    diagnostics: Vec<Diagnostic>, // syntax warnings/errors
    mtime: SystemTime,
}

/// A syntax issue
struct Diagnostic {
    level: DiagnosticLevel,   // Error | Warning
    message: String,
    file: PathBuf,
    line: u32,
    column: u32,
}

enum DiagnosticLevel {
    Error, // malformed syntax (REQ-007)
}

```



```
Warning, // ambiguous resolution, potential issues
}
```

6. Contract Specifications (CLI Interface)

All commands operate on the vault directory, defaulting to the current working directory.

6.1 Global Flags

CON-001: **Global CLI Interface**

zetl [OPTIONS] <COMMAND>

Options:

-d, --dir <PATH>	Vault root directory [default: .]
-f, --format <FORMAT>	Output format : json table [default: json]
--no-cache	Force full rescan, ignore cached index
--no-color	Disable colored output
-q, --quiet	Suppress non-essential output
-v, --verbose	Increase verbosity (repeat for more : -vv)
-h, --help	Print help
-V, --version	Print version

Implements:

- REQ-001, REQ-011, REQ-012

Verified by:

- TEST-001, TEST-011, TEST-012

6.2 Subcommands

CON-002: **index**

zetl index [OPTIONS]

Build or refresh the link index for the vault.

Behaviour:

- Scans all *.md files in the vault directory (recursive)
- Parses wikilinks, builds graph, writes cache to .zetl/index.json
- Reports: files scanned, links found, time elapsed

Exit codes:

0	Success
1	Fatal error (unreadable directory, permission denied)

Example output (JSON):

```
{
  "files_scanned": 1423,
  "links_found": 8291,
  "dead_links": 12,
  "diagnostics": 3,
  "elapsed_ms": 487
}
```

Implements:

- REQ-001, REQ-002

Verified by:
- TEST-001, TEST-002

CON-003: links / backlinks

zetl links <PAGE> [OPTIONS]
zetl backlinks <PAGE> [OPTIONS]

Query forward links from a page, **or** backlinks to a page.

Arguments:
 <PAGE> Page name (case-insensitive, partial match **with** --fuzzy)

Options:
 --fuzzy Enable fuzzy page name matching
 --context <N> Include N characters of surrounding text [default: 0]
 --depth <N> Traverse N hops (1 = direct only) [default: 1]

Exit codes:
 0 Results found
 1 Page not found (suggest similar names)

Example output (JSON, backlinks):

```
{
  "page": "Zettelkasten Method",
  "backlinks": [
    {
      "source": "Knowledge Management.md",
      "line": 14,
      "context": "...inspired by the [[Zettelkasten Method]] developed by...",
      "alias": null,
      "is_embed": false
    }
  ],
  "count": 1
}
```

Implements:
- REQ-003, REQ-004

Verified by:
- TEST-003, TEST-004

CON-004: check

zetl check [OPTIONS]

Validate the vault: **report** dead links, orphans, and **syntax** errors.

Options:
 --dead-links Show only dead links
 --orphans Show only orphan pages
 --syntax Show only **syntax** errors
 --fail-on <LVL> **Exit** non-zero if issues at level: **error** | warning [default: **error**]

Exit codes:
 0 No issues at or above --fail-on level
 1 Issues found
 2 Fatal **error**

Example output (JSON):

```
{
  "dead_links": [
    {
      "source": "README.md",
      "line": 7,
```

```

    "target": "Nonexistent Page"
  }
],
"orphans": [
  {
    "page": "Stale Draft.md",
    "forward_links": 2
  }
],
"syntax_errors": [
  {
    "file": "Notes.md",
    "line": 42,
    "column": 10,
    "message": "Unclosed wikilink: '['[missing bracket'"
  }
],
"summary": {
  "dead_links": 1,
  "orphans": 1,
  "syntax_errors": 1
}
}

```

Implements:

– REQ-005, REQ-006, REQ-007

Verified by:

– TEST-005, TEST-006, TEST-007

CON-005: similar

zectl similar <QUERY> [OPTIONS]

Find pages **with** names similar to the query using SimHash + Hamming distance.

Arguments:

<QUERY> Search string

Options:

--threshold <N> Max Hamming distance [default: 3]

--limit <N> Max results [default: 10]

Exit codes:

0 Results found

1 No matches within threshold

Example output (JSON):

```

{
  "query": "zettelkasen",
  "results": [
    {
      "page": "Zettelkasten Method",
      "distance": 2,
      "path": "notes/Zettelkasten Method.md"
    },
    {
      "page": "Zettelkasten History",
      "distance": 3,
      "path": "notes/Zettelkasten History.md"
    }
  ]
}

```

Implements:

– REQ-008

Verified by:

– TEST-008

CON-006: stats

zetl stats [OPTIONS]

Print summary statistics about the vault's link graph.

Options:

--top <N> Number of most-linked pages to show [default: 10]

Example output (JSON):

```
{
  "pages": 1423,
  "links": 8291,
  "unique_targets": 1108,
  "dead_links": 12,
  "orphans": 47,
  "ambiguous_links": 3,
  "connected_components": 5,
  "most_linked": [
    { "page": "Index", "backlink_count": 312 },
    { "page": "Zettelkasten Method", "backlink_count": 89 }
  ]
}
```

Implements:

– REQ-009

Verified by:

– TEST-009

CON-007: path

zetl path <FROM> <TO> [OPTIONS]

Find the shortest link path between two pages.

Arguments:

<FROM> Source page name

<TO> Target page name

Options:

--max-depth <N> Maximum path length to search [default: 10]

Exit codes:

0 Path found

1 No path found (pages in different components or not found)

Example output (JSON):

```
{
  "from": "Rust Programming",
  "to": "Category Theory",
  "hops": 3,
  "path": [
    "Rust Programming",
    "Type Systems",
    "Lambda Calculus",
    "Category Theory"
  ]
}
```

Implements:

– REQ-010

Verified by:
– TEST-010

7. SimHash Design

7.1 Algorithm

SimHash produces a fixed-width fingerprint (64-bit) for a text string such that similar strings produce fingerprints with low Hamming distance.

Procedure for a page name:

1. **Normalize:** lowercase, collapse whitespace, strip punctuation
2. **Tokenize:** extract character trigrams (sliding window of 3)
 - "zettelkasten" → {"zet", "ett", "tte", "tel", "elk", "lka", "kas", "ast", "ste", "ten"}
3. **Hash each trigram:** using a fast, uniform hash (e.g., FNV-1a) to a 64-bit value
4. **Accumulate:** for each bit position 0..63, sum +1 if the trigram hash has a 1, else -1
5. **Threshold:** set each bit to 1 if the sum > 0, else 0
6. **Result:** a 64-bit fingerprint

Hamming distance between two fingerprints = `(a XOR b).count_ones()`. A distance of 0 means identical fingerprints; ≤ 3 indicates high similarity.

7.2 Search Strategy

For vaults up to ~10,000 pages, brute-force scan of all fingerprints is fast enough (< 1ms). For larger vaults, a multi-probe table or bit-permutation index may be added in a future SPEC.

7.3 Use Cases

- **Typo detection in links:** `[[Zettelkasen]]` → did you mean `[[Zettelkasten Method]]` ?
- **Duplicate avoidance:** before creating a new page, check if a similar one exists
- **Suggested links:** find pages with similar names that might be worth linking

8. Test Specifications

TEST-001: Index Scan Completeness

Scenario: Scan a test vault with known files and links
Given: A directory with 5 Markdown files containing 12 wikilinks
When: `zetl index` is run
Then:

- All 5 files are indexed
- All 12 wikilinks are extracted with correct file, line, column
- Links inside code blocks and comments are excluded
- Elapsed time is reported

Verifies: REQ-001

TEST-002: Graph Construction

Scenario: Build graph from parsed files

Given: File A links to B and C; File B links to C; File D has no links

When: The graph is constructed

Then:

- Graph has 4 nodes (A, B, C, D)
- Graph has 3 edges (A→B, A→C, B→C)
- Backlinks for C = [A, B]
- D is an orphan (zero incoming edges)

Verifies: REQ-002

TEST-003: Forward Link Query

Scenario: Query forward links for a page

Given: An indexed vault where "Index.md" contains [[Page A]], [[Page B|alias]]

When: `zetl links Index` is run

Then:

- Returns 2 links
- Page A link has no alias
- Page B link has alias "alias"
- Output matches CON-003 schema

Verifies: REQ-003

TEST-004: Backlink Query

Scenario: Query backlinks for a target page

Given: An indexed vault where 3 files link to "Concept X"

When: `zetl backlinks "Concept X"` is run

Then:

- Returns 3 backlinks with correct source files and line numbers
- Context text (if --context used) includes surrounding characters

Verifies: REQ-004

TEST-005: Dead Link Detection

Scenario: Detect links to non-existent pages

Given: File A contains [[Existing Page]] and [[Ghost Page]]

Only "Existing Page.md" exists

When: `zetl check --dead-links` is run

Then:

- Reports 1 dead link: "Ghost Page" from File A with line number
- Exit code 1

Verifies: REQ-005

TEST-006: Orphan Detection

Scenario: Find pages with no backlinks

Given: 4 files exist; File D is never referenced by any other file

When: `zetl check --orphans` is run

Then:

- File D appears in the orphan list
- Files A, B, C do not appear (they have at least one backlink)

Verifies: REQ-006

TEST-007: Syntax Validation

Scenario: Detect malformed wikilinks

Given: A **file** containing:

Line 5: "See [[unclosed bracket"

Line 8: "Empty [[]] link"

Line 12: "Valid [[Good Link]]"

When: `zetl check --syntax` is run

Then:

- Reports 2 diagnostics (lines 5 and 8)
- Line 12 produces **no** diagnostic
- Diagnostic messages **describe** the specific issue

Verifies: REQ-007

TEST-008: SimHash Fuzzy Search

Scenario: Find similar page names

Given: Pages named "Zettelkasten Method", "Zettelkasten History", "Rust Programming"

When: `zetl similar "zettelkasen"` is run with threshold 3

Then:

- "Zettelkasten Method" appears (distance ≤ 3)
- "Zettelkasten History" appears (distance ≤ 3)
- "Rust Programming" does not appear

Verifies: REQ-008

TEST-009: Graph Statistics

Scenario: Summary stats for a vault

Given: A vault with known file/link counts

When: `zetl stats` is run

Then:

- All reported counts match expected values
- most_linked list is sorted descending by backlink_count
- connected_components count is correct

Verifies: REQ-009

TEST-010: Shortest Path

Scenario: Find shortest path between two pages

Given: A→B→C→D (linear chain); A→D does **not** exist

When: `zetl path A D` is run

Then:

- Returns path ["A", "B", "C", "D"] with hops=3

Scenario: No path exists

Given: E is in a separate component

When: `zetl path A E` is run

Then:

- Exit code 1, message "no path found"

Verifies: REQ-010

TEST-011: Cache Behaviour

Scenario: Cache speeds up subsequent runs

Given: A vault indexed once (cache written to .zetl/index.json)

When: `zetl index` is run again with no file changes

Then:

- Completes **in ≤ 50% of** the initial scan time
- Produces identical results

Scenario: Cache invalidation **on** file change

Given: A cached vault **where** one file **is** modified

When: ``zetl index`` **is** run

Then:

- Only the modified file **is** re-parsed
- Results reflect the updated content

Verifies: REQ-011

TEST-012: Ignore Patterns

Scenario: Excluded paths are **not** scanned

Given: A vault **with** `.zetlignore` containing `"drafts/"`

When: ``zetl index`` **is** run

Then:

- Files under `drafts/` are **not** indexed
- `.git` **and** `node_modules` are excluded **by default**

Verifies: REQ-012

9. Observability

OBS-001: Index Timing

The CLI SHALL emit **timing** information (`elapsed_ms`) **for** the `index` command **to** support performance monitoring **over time**.

OBS-002: Verbose Logging

When `--verbose` *is specified*, the CLI SHALL emit *per-file scanning progress* **and** link resolution decisions **to** `stderr`.

10. Future Considerations

These items are explicitly **out of scope** for SPEC-001 but are anticipated for future SPECS:

Item	Rationale
Tag extraction (<code>#tag</code> , YAML frontmatter)	Natural extension; same scanner infrastructure
Graph export (DOT, Mermaid, GraphML)	Visualization for humans; low effort once graph exists
Watch mode (<code>zetl watch</code>)	Re-index on file change for long-running agent sessions
MCP server mode	Expose <code>zetl</code> as a Model Context Protocol tool server
Block-level graph	Track <code>^block-id</code> references as first-class graph nodes

Item	Rationale
Embedding-based similarity	Upgrade from SimHash to vector embeddings for deeper semantic search
Multi-vault support	Index across multiple directories with namespace prefixes

11. Traceability Matrix

REQ	CON	TEST	OBS
REQ-001	CON-001, CON-002	TEST-001	OBS-001
REQ-002	CON-002	TEST-002	—
REQ-003	CON-003	TEST-003	—
REQ-004	CON-003	TEST-004	—
REQ-005	CON-004	TEST-005	—
REQ-006	CON-004	TEST-006	—
REQ-007	CON-004	TEST-007	—
REQ-008	CON-005	TEST-008	—
REQ-009	CON-006	TEST-009	—
REQ-010	CON-007	TEST-010	—
REQ-011	CON-001	TEST-011	—
REQ-012	CON-001	TEST-012	—

12. Open Questions

1. Should `zetl` support Logseq's `[alias]([page])` syntax as an alternative parser mode? Logseq uses this in addition to standard `[[wikilinks]]`. Adding it would improve compatibility but increases parser complexity. Recommendation: defer to a future SPEC unless users report demand.
2. Should heading/block references create separate graph edges or just metadata on the page edge? Current design treats them as metadata. Promoting them to first-class nodes would enable finer-grained queries but significantly increases graph size.
3. What is the right default for `--context` in backlink queries? Zero (no context) is cleanest for agents; 80 characters is more useful for humans. Current default is 0 with the flag available.